2 0

LANGUAGE • SYNTAX • APIS

— Intro

Explanation of Proxy and Reflect API's that JavaScript offers. Let's investigate the typical use cases of each and understand how they work!

Technical check

Author

Linguistic check

— Prelude

Have you ever wondered how to interact with code written by other programmers without intruding on their code?

There is something called **metaprogramming**. Thanks to it you can achieve it.

Two objects will help you: **Proxy** and **Reflect**. Today you will learn how, when, and where you can use them to master a powerful technique.

> *Metaprogramming is not invented by **Meta** company 🤓.*

1. Metaprogramming

To start talking about metaprogramming, you need to answer the question: what is data and code? Every programming language has both of these but what do they mean, exactly?

Data can be for example:

- number
- symbol
- array
- object
- string

Code is an expression - an instruction to be executed on this data. It can be: `const result = 1 + (2 * 3)` .

At first glance, you can say that it is a mathematical expression, and this is indeed the case if we are talking about the field of mathematics. However, according to the presented approach, we will consider it in two ways:

- by data: '1 + (2 * 3)' - is a string!
- by code: '1 + (2 * 3)' - is it a code snippet? Then it is 7!

From this it follows that the same representation of an expression can have different interpretations. This approach is what metaprogramming is all about.

Knowing this is a powerful tool for building data and expressions. It allows us to create functions that multiply data which means we can write programs that write programs. It is a certain level of abstraction.

To illustrate this more easily, let's discuss it with this example:

Copy

```
for (let i = 0; i < 10; i++) {
  if (i > 5) {
    // Do something.
  }
}
```

For loop

According to the data it is a plain text, but according to the code it is something to be interpreted as an instruction, so:

Copy

```
(for from 0 to 10) -> block(
    SET i
    CHECK i > 5
        THEN /* Do something. */
)
```

For as set of instructions

Congratulations, you have just successfully interpreted a piece of code through metaprogramming!

2. Proxy pattern explained

A **proxy** is a **structural design pattern** that allows you to create a proxy object in place of another object. A proxy is a kind of guardian that protects access to the original object, or changes its behavior.

How do you translate this into a simple example from life?... Cash! Cash is the physical representation of the capital placed in your bank account. But what about it? Have you ever used a credit card?

Well, think about it. Credit card is an actually proxy for your cash in the bank. You don't need to physically have paper money with you to buy something. Both of these entities implement the same interface (you can perform an action with them - pay). At the same time, you are actually operating an object that protects another object (in the bank).

3. Proxy in JavaScript

Let's start with a simple example for reference:

Copy

```
const ProtectedObject = {
  name: "John",
  last: "Smith",
  age: 30,
}

const Handler = {
  get: function (target, property) {
    if (property.includes("_")) {
      return property
        .split("_")
        .map(prop => `${prop}: ${target[prop]}`)
        .join("\n")
    }

    return target[property]
  },
}

const ProxyProtectedObject = new Proxy(ProtectedObject, Handler)

console.log(ProxyProtectedObject.name_last_age) // name: John, last: Smith, age:
```

Your first proxy

As you can see, we have overwritten the **get** mechanism for reading the properties for the **ProtectedObject** without interfering with its implementation. Now reading a property separated by the symbol "_", you are able to transform several properties into a string of characters.

Even more can be achieved by overwriting **set** behavior. Just look at this:

Copy

```
"use strict" // <- Strict mode on.

const form = {
  name: "",
}
```

```
6
7  const proxyForm = new Proxy(form, {
8    set: function (obj, prop, value) {
9      if (prop in obj) {
10       obj[prop] = value
11       return true
12     } else {
13       return false
14     }
15   },
16 })
17
18 proxyForm.age = 20 // Uncaught TypeError: 'set' on proxy (form object does not ha
19 proxyForm.name = "Jaroslaw" // That's ok!
```

Strict

We added validation which makes sense. Who normally tries to add something to an object with a certain shape?

> *When does a proxy make sense? The proxy should totally replace the target object everywhere. No one should ever reference the target object after it got proxied. Otherwise it's easy to mess up.*

4. Proxy use cases

Logging access to an object:

✂️ Copy

```
1  const obj = {
2    name: "Jonh",
3    last: "Smith",
4  }
5
6  const objProxy = new Proxy(obj, {
7    get: function (obj, prop) {
8      /** Send information to another module. */
9      console.log(`${prop} readed`)
10
11     return obj[prop]
12   },
```

```
13 })
14
15 const nameRead = objProxy.name // Name readed.
```

Logging properties

Custom error handling/validation:

Copy

```
1  const form = {
2    name: "",
3    last: "",
4    age: null,
5  }
6
7  const proxyForm = new Proxy(form, {
8    set: function (obj, prop, value) {
9      if (prop === "age" && value < 0) {
10       throw Error(`Age cannot be ${value}, is less than 0!`)
11     }
12
13     if (prop in obj) {
14       obj[prop] = value
15       return true
16     } else {
17       return false
18     }
19   },
20 })
21
22 proxyForm.age = -1 // Uncaught Error: Age cannot be -1, is less than 0!
23 proxyForm.age = 2
24
25 console.log(proxyForm.age) // 2
```

Cool validation

Protect the object field:

Copy

```
1  const form = {
2    nick: "",
```

```
  pass: "",
}

const proxyForm = new Proxy(form, {
  get: function (obj, prop) {
    if (prop === "pass") {
      return undefined
    }

    if (prop in obj) {
      return obj[prop]
    }

    return undefined
  },
})

proxyForm.nick = "john"
proxyForm.pass = "qwerty"

console.log(proxyForm.pass) // undefined
```

Like a guard

Lazy initialization:

Copy

```
class Server {
  toString() {
    return "HEAVY!!!"
  }
}

const handle = {}
console.log(handle["server"]) // undefined

class LazyInit {
  get(target, prop, receiver) {
    if (this.server === null) {
      this.server = new Server()
    }
    return this.server
  }

```

```
  server = null
}

const handle = new Proxy(Server, new LazyInit())
console.log(handle.server.toString()) // HEAVY!!!
```

Lazy

5. What is Reflection?

Reflection is a closely related concept to object-oriented programming. Among other things, it is used to check the properties of a **class/object/method**. This technique allows building more dynamic functionality that is native to a particular programming language.

Languages that use this functionality are able to modify their own environment. So the use of reflection should be thoughtful and used only in specific cases to avoid messy effects.

6. Reflect feature in JavaScript

Reflect is a Javascript native object that provides methods same as proxy feature. It is not a functional object, so its methods are static (not constructable). What does this mean? We can't create an instance of a Reflect object through the **new** operator or call it like a function.

So which approach should I use? In my opinion: **both**.

Proxy provides us with a number of options for changing an object's behavior during particular actions, while Reflect gives us access to their default behavior.

It is a modified example "Protect the object field":

✂️ Copy

```
const form = {
  nick: "",
  pass: "",
}

const proxyForm = new Proxy(form, {
  get: function (obj, prop) {
    if (prop === "pass") {
      return undefined
    }
```

```
11
12      return Reflect.get(obj, prop)
13    },
14  })
15
16  proxyForm.nick = "john"
17
18  console.log(proxyForm.nick)
```

Guard but with Reflect

Simpler? Clearer? Without an additional condition? Of course it is.

Besides, the Reflect object provides us with a lot of functionality, which can be categorized as "syntax sugar".

7. Reflect use cases

Need to find out if an object has a field?

✂ Copy

```
1  Reflect.has(object, field) //-> booelan
```

Searching...

Need to return all the keys of an object?

✂ Copy

```
1  Reflect.ownKeys(object) // -> string[]
```

Keys list

Want to add a new field? No problem!

✂ Copy

```
1  Reflect.set(object, prop, value) //-> boolean, returns true if successful.
```

Adding field

Summary

I hope you now understand Proxy and Reflect 🤦. I must say, they are tricky, yet useful and powerful.

Next time when you see an ugly, old legacy code remember that you have these API's and there may be a good use case for them.

Feel free to contact me if you have any questions/proposals. Have a nice day and good health!

Kacper Woźniak
Frontend developer

I create content regularly!

I hope you found my post interesting. If so, maybe you'll take a peek at my LinkedIn, where I publish posts daily.

FOLLOW ME

Comments

Add your honest opinion about this article and help us improve the content.

2 0

CREATED: 06-01-2023 UPDATED: 06-01-2023

SHARE

SHOW SOURCE

PREVIOUS